

Operator Splitting for Optimal Control

Boyer Timothé, Hacini Malik, Lainé Martin, Mouret Basile

April 22, 2026

What's optimal control?

Optimal control is about choosing **inputs** for a **dynamical system** so that it behaves as desired while **minimizing a cost**.

In many applications, the state evolves over time and each decision affects both the immediate cost and the future trajectory. This temporal coupling is what makes the problem both useful and difficult.



What's optimal control?

Goal: keep the indoor temperature close to 20°C over time. Here, the **state** is the room temperature T , and the **control** is the power P sent to the radiator.

$$T_{t+1} = f(T_t, P_t)$$

- ▶ We minimize thermal discomfort and electricity usage.
- ▶ We respect the limits: radiator power cannot go below 0 W or exceed a maximum value.

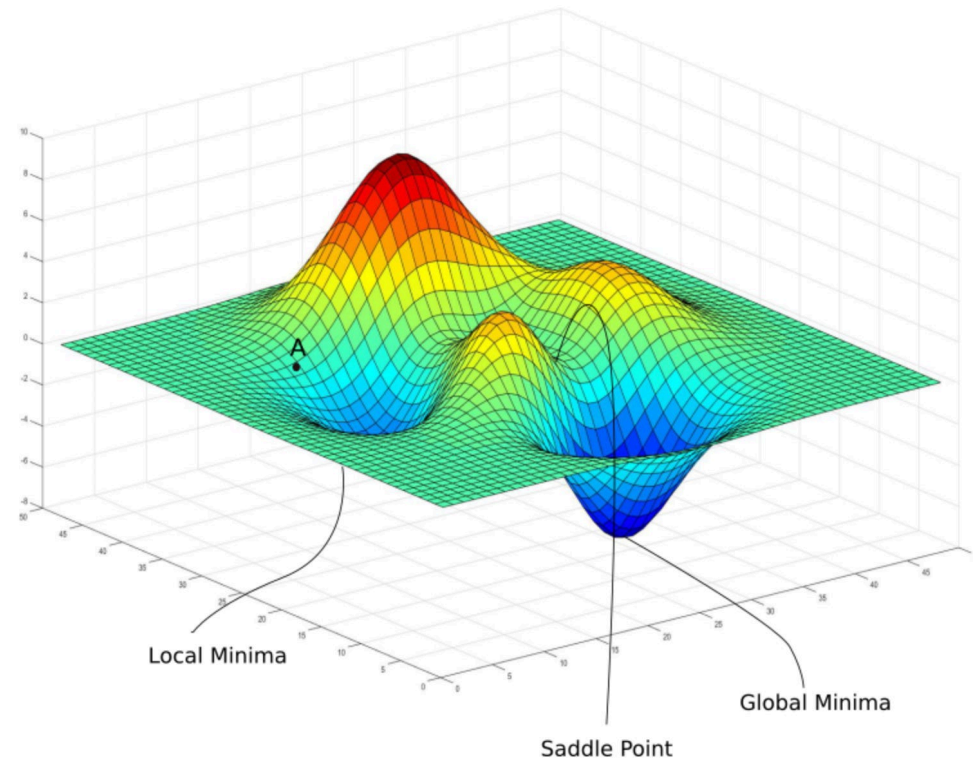
Optimal control finds the best heating sequence to balance comfort and cost under these limits.



Why the splitting method?

We used the method proposed by O'Donoghue, Stathopoulos, and Boyd because it is designed to exploit this problem structure directly.

Alternative approaches such as CVX or fast MPC can also be used on related optimal-control problems, but OSC (operator splitting for control) is especially well suited to our setting.



Context

We consider a finite-horizon problem with linear dynamics and a stage cost split into a smooth quadratic part and a convex nonsmooth part.

$$\min_{x,u} \sum_{t=0}^T (\varphi_t(x_t, u_t) + \psi_t(x_t, u_t))$$

$$\text{subject to } x_{t+1} = A_t x_t + B_t u_t + c_t, \quad t = 0, \dots, T-1, x_0 = x_{\text{init}}.$$

Where φ_t is quadratic and ψ_t can encode nonsmooth penalties or convex constraints. We respectively denote $\varphi(x, u)$ and $\psi(x, u)$ as the total quadratic and convex costs over time.

Consensus splitting

We introduce a copy $(\tilde{x}, \tilde{u})$ of the trajectory:

$$\begin{aligned} \min_{x, u, \tilde{x}, \tilde{u}} \quad & I_D(x, u) + \varphi(x, u) + \psi(\tilde{x}, \tilde{u}) \\ \text{subject to} \quad & (x, u) = (\tilde{x}, \tilde{u}). \end{aligned}$$

This does not change the solution. It only separates the objective into two blocks. The dynamics stay in the (x, u) block through I_D , so the $(\tilde{x}, \tilde{u})$ block has no cross-time constraints.

Consensus splitting

We introduce a copy $(\tilde{x}, \tilde{u})$ of the trajectory:

$$\begin{aligned} \min_{x, u, \tilde{x}, \tilde{u}} \quad & I_D(x, u) + \varphi(x, u) + \psi(\tilde{x}, \tilde{u}) \\ \text{subject to} \quad & (x, u) = (\tilde{x}, \tilde{u}). \end{aligned}$$

This does not change the solution. It only separates the objective into two blocks. The dynamics stay in the (x, u) block through I_D , so the $(\tilde{x}, \tilde{u})$ block has no cross-time constraints.

This allows us to solve the problem with ADMM, an algorithm for decomposable optimization problems.

ADMM updates

The method alternates between three simple operations:

$$(x^{k+1}, u^{k+1}) = \arg \min_{x, u} I_D(x, u) + \varphi(x, u) + \frac{\rho}{2} \|(x, u) - (\tilde{x}^k, \tilde{u}^k) - (z^k, y^k)\|_2^2$$

$$(\tilde{x}^{k+1}, \tilde{u}^{k+1}) = \arg \min_{\tilde{x}, \tilde{u}} \psi(\tilde{x}, \tilde{u}) + \frac{\rho}{2} \|(\tilde{x}, \tilde{u}) - (x^{k+1}, u^{k+1}) + (z^k, y^k)\|_2^2$$

$$(z^{k+1}, y^{k+1}) = (z^k, y^k) + (\tilde{x}^{k+1}, \tilde{u}^{k+1}) - (x^{k+1}, u^{k+1}).$$

The first step is a quadratic control problem with linear constraints, and the second is exactly a proximal update.

Here, ρ controls the strength of the quadratic penalty that keeps the new variables close to the current ADMM point.

ADMM updates

The method alternates between three simple operations:

$$(x^{k+1}, u^{k+1}) = \arg \min_{x, u} I_D(x, u) + \varphi(x, u) + \frac{\rho}{2} \|(x, u) - (\tilde{x}^k, \tilde{u}^k) - (z^k, y^k)\|_2^2$$

$$(\tilde{x}^{k+1}, \tilde{u}^{k+1}) = \arg \min_{\tilde{x}, \tilde{u}} \psi(\tilde{x}, \tilde{u}) + \frac{\rho}{2} \|(\tilde{x}, \tilde{u}) - (x^{k+1}, u^{k+1}) + (z^k, y^k)\|_2^2$$

$$(z^{k+1}, y^{k+1}) = (z^k, y^k) + (\tilde{x}^{k+1}, \tilde{u}^{k+1}) - (x^{k+1}, u^{k+1}).$$

The first step is a quadratic control problem with linear constraints, and the second is exactly a proximal update.

Here, ρ controls the strength of the quadratic penalty that keeps the new variables close to the current ADMM point. We can therefore solve these two problems independently with the method best suited to each one. This justifies the core “splitting” idea of the method.

Solving the subproblems

The first subproblem keeps the horizon coupling, but it is purely quadratic with linear constraints. After stacking all states and controls, it reduces to one structured KKT system:

$$\begin{pmatrix} E & G^T \\ G & 0 \end{pmatrix} \begin{pmatrix} w \\ \lambda \end{pmatrix} = \begin{pmatrix} -f \\ h \end{pmatrix}.$$

Solving the subproblems

The first subproblem keeps the horizon coupling, but it is purely quadratic with linear constraints. After stacking all states and controls, it reduces to one structured KKT system:

$$\begin{pmatrix} E & G^T \\ G & 0 \end{pmatrix} \begin{pmatrix} w \\ \lambda \end{pmatrix} = \begin{pmatrix} -f \\ h \end{pmatrix}.$$

The second subproblem separates across time:

$$(\tilde{x}_t^{k+1}, \tilde{u}_t^{k+1}) = \text{prox}_{\frac{\psi_t}{\rho}}(x_t^{k+1} - z_t^k, u_t^{k+1} - y_t^k).$$

In many cases, the proximal operator is available in closed form.

Julia Implementation



- ▶ Designed for scientific computing
- ▶ High-level syntax
- ▶ Just-in-time compiled

Solver API

```
using LinearAlgebra
using OptimalControl_OperatorSplitting

function prox!(x_tilde, u_tilde, v, w, rho)
    x_tilde .= v
    u_tilde .= clamp.(w, umin, umax)
end

data = all_data(A, B, c, Q, S, R, q, r, x_init)

cache = setup_cache(data)

x, u, tt = solve(cache, prox!; max_iters=3000)
```

Live Demo

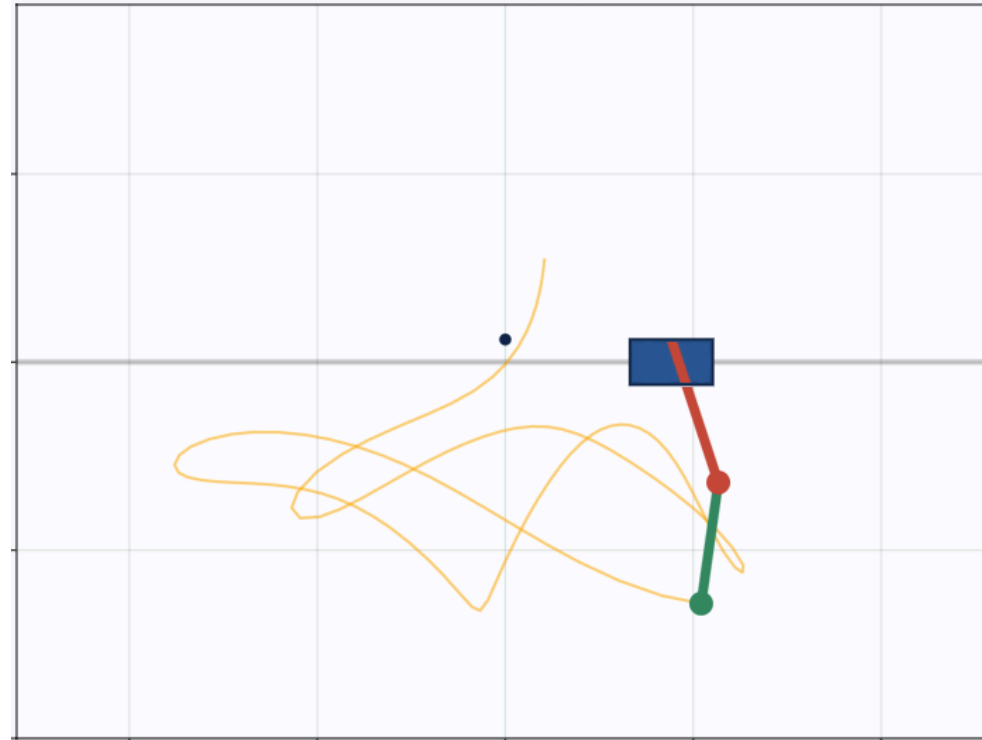


Figure: Double Pendulum controlled by a cart

Results

Example: Box-constrained quadratic optimal control

$$\min_{x,u} \frac{1}{2} \sum_{t=0}^{T-1} (x_t^T Q x_t + u_t^T R u_t)$$

For the speed-control example, we impose

- ▶ $x_{t+1} = Ax_t + Bu_t$
- ▶ $x_0 = x_{\text{init}}$
- ▶ $|u_t| \leq 1$

Results

Example: Box-constrained quadratic optimal control

$$\min_{x,u} \frac{1}{2} \sum_{t=0}^{T-1} (x_t^T Q x_t + u_t^T R u_t)$$

For the speed-control example, we impose

- ▶ $x_{t+1} = Ax_t + Bu_t$
- ▶ $x_0 = x_{\text{init}}$
- ▶ $|u_t| \leq 1$

Speed control example

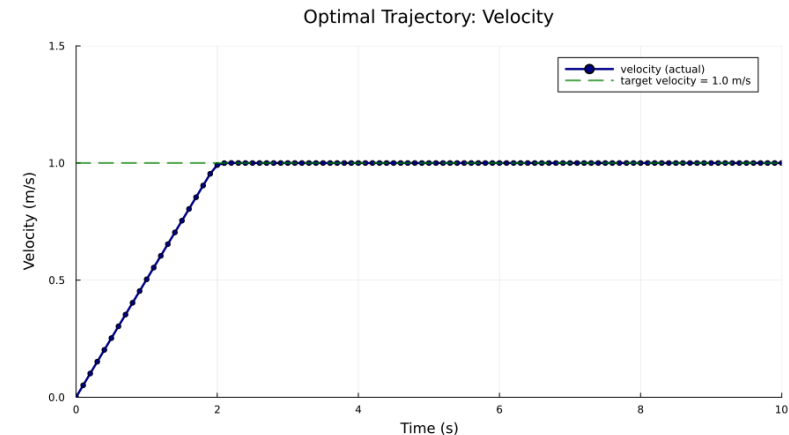
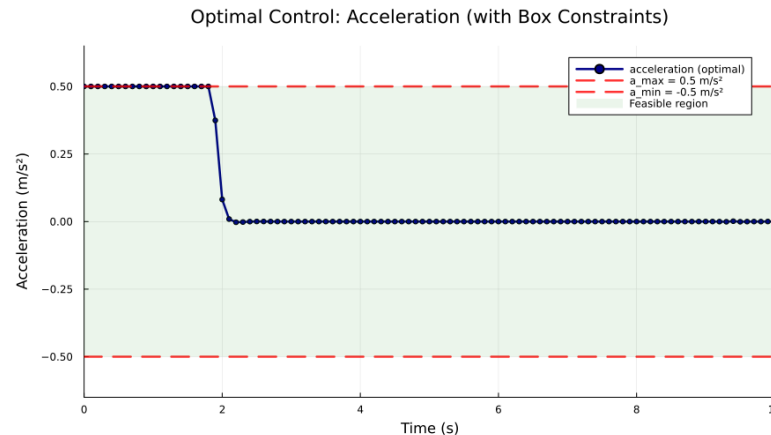


Figure: Acceleration (left) and velocity (right) for the controlled trajectory.

Performance comparison against C

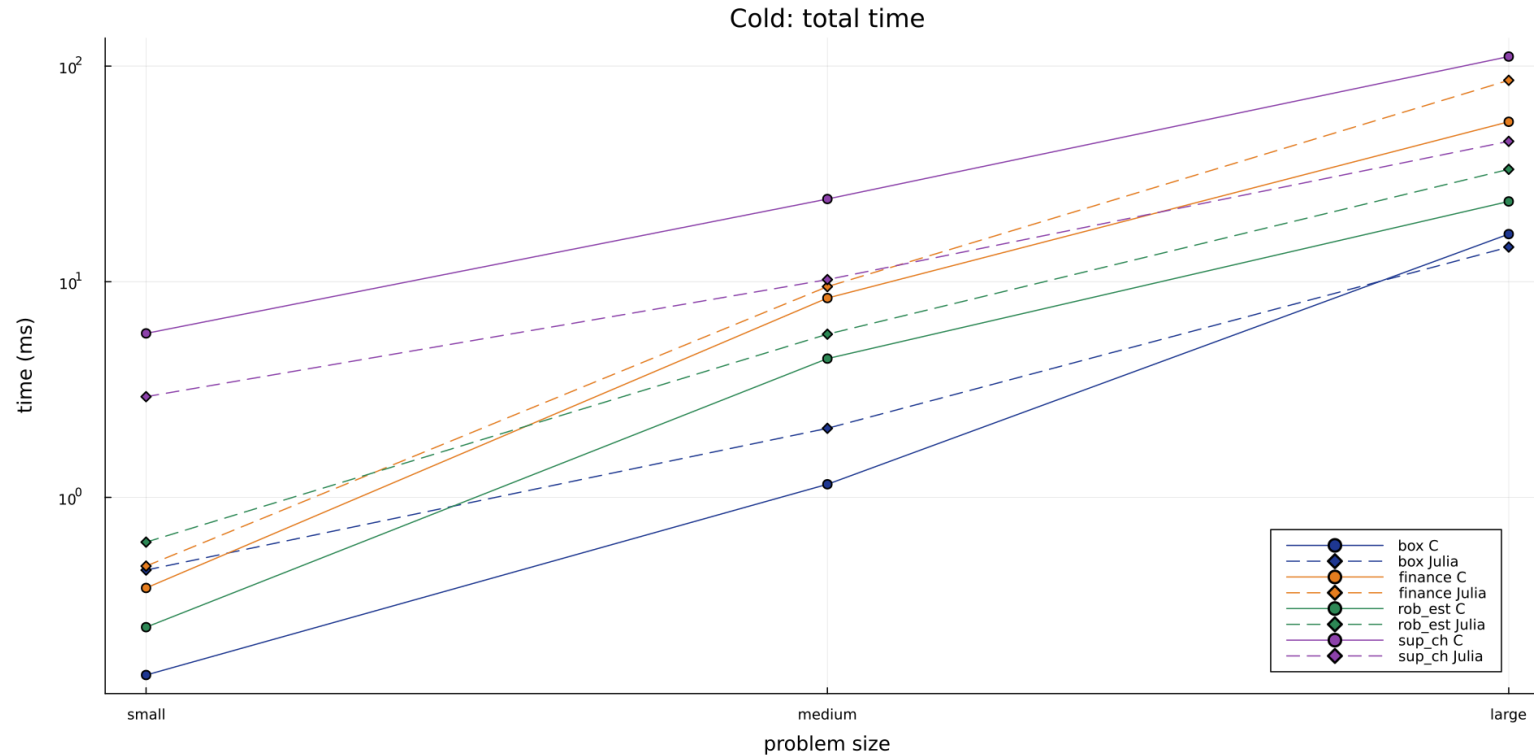


Figure: Execution time of the algorithm for the different problems in the cold-start case.

Performance comparison against C

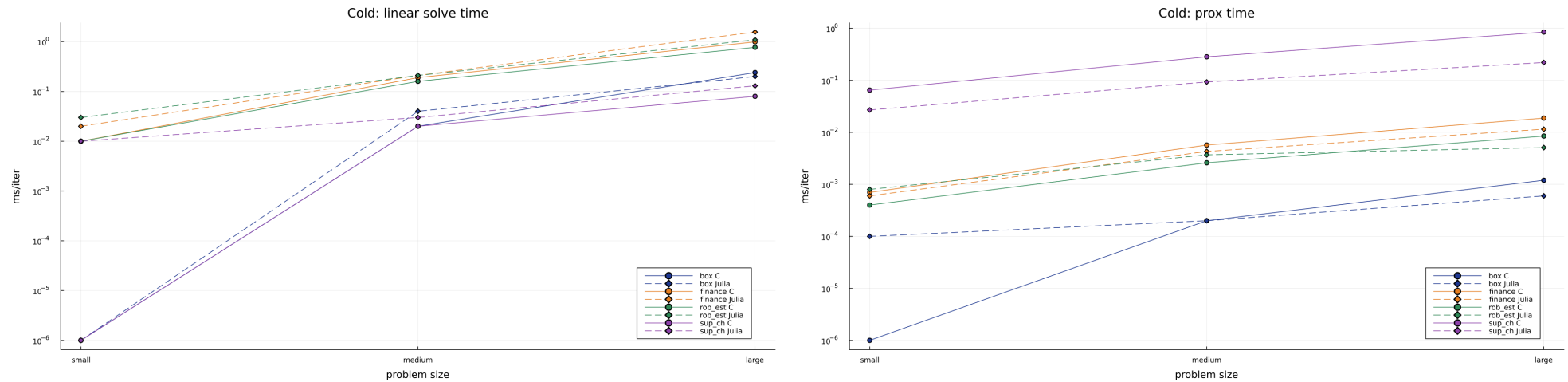


Figure: Execution time of the linear solve (left) and the proximal update (right) for the different problems in the cold-start case.

Recap and Future work

We implemented the operator-splitting method in Julia for finite-horizon optimal control with linear dynamics.

On the benchmark examples, the Julia solver matches the reference C solver in iteration counts. Runtime differences depend on the example and on the relative cost of the linear-system step versus the proximal step.

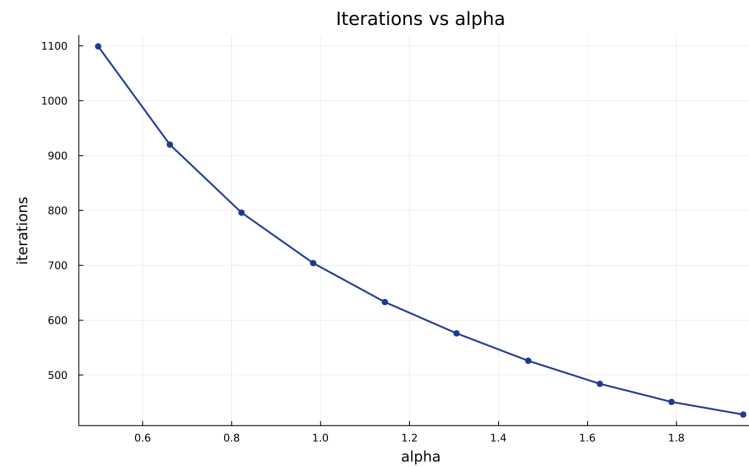
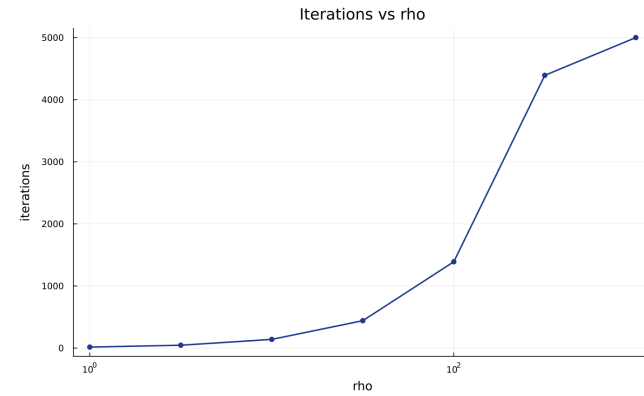
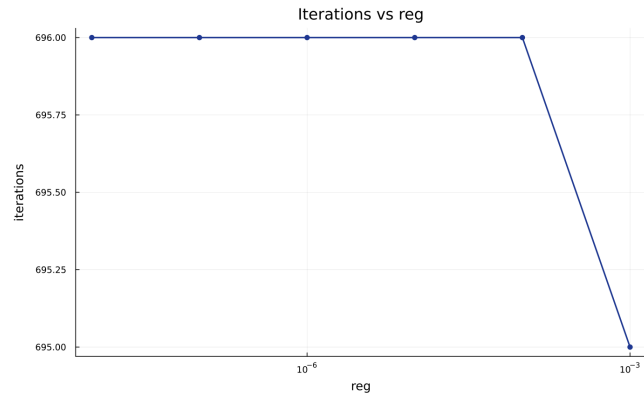
Possible extensions :

- ▶ Using a native Julia linear solver
- ▶ Test on more examples
- ▶ Convergence-rate results

Conclusion

Thank you for your attention !

Hyperparameter sensitivity



Stopping criterion sensitivity

